

## What the project does

This project implements a distributed trading bot deployed across a network on the FABRIC testbed. Designed using a microservices architecture, the system separates concerns into three distinct network nodes communicating with ZeroMQ:

1. Ingestion Node (n1): Acts as the data stream provider. It establishes a WebSocket connection to the Binance testnet API, listens for live tick data, and broadcasts this data continuously over a ZeroMQ PUB socket.
2. Decision Node: It connects to the ingestion stream with a ZeroMQ SUB socket, parses the live market data, requests historical data from yfinance API, and evaluates it against a trading strategy. When a trade condition is met, it formats an order payload and sends it downstream via a ZeroMQ PUSH socket. For each asset, creating a new node for each will isolate each decision and resources it will need. So scaling up the amount of assets to trade will also need to increase the amount of fabric nodes.

The trading strategy is made from Simple Moving Average and Relative Strength Index. Simple Moving Average calculates the average closing price over x amount of days. Relative Strength Index calculates the changes of price and has a value of 0 to 100. SMA over 14 days represents the short term gains when the market shifts. SMA over 50 days represents the medium term gains and requires a sustained price change. For example, if the current price is above the 14 day SMA and less than 50 day SMA, that will indicate that the current price will likely bounce downwards. When the current price is above 14 and 50 day SMA, that will indicate that the current price will likely keep trending upwards. When the RSI is greater than 70, it indicates that it is overbought and oversold when it is less than 30.

To trigger a buy action, the previous price must be less than or equal to a 50 day SMA, and current price must be greater than 50 day SMA and have RSI be less than 70. By including the previous price, it prevents it from duplicating buy actions.

To trigger a sell action, the previous price must be greater than or equal to a 10 day SMA and current price must be less than a 10 day SMA. This is designed to sell when it indicates a downward trend. When selling, it is not looking for profit, it is minimizing risk.

$$n \text{ day SMA} = \frac{P_1 + P_2 + \dots + P_n}{n}, \text{ } n = \text{amount of days, and } P \text{ is price}$$

$$RSI = 100 - \frac{100}{(1+RS)}, RS = \frac{AVGU}{AVGD}, AVGU = \text{Average of all up moves in } N \text{ bars}$$

$AVGD = \text{Average of all down moves in } N \text{ bars, } N = \text{period}$

3. Execution Node (n3): Acts as the broker interface. It listens for incoming trade signals on a ZeroMQ PULL socket and executes Market Orders (Buy/Sell)

## How to use the implementation

### 1. Creating nodes

The system requires three Ubuntu nodes deployed and wired together on an IPv4 network. Run the following deployment script in a FABRIC Jupyter environment to create the slice and configure the network interface cards.

```
from fabrictestbed_extensions.fablib.fablib import FablibManager as fablib_manager

fablib = fablib_manager(project_id="f5b4fc7d-978f-45be-9530-b38db8ef5046")
slice_name = 'cs451_project_v3'
image = 'default_ubuntu_22'
site = "STAR"
slice = fablib.new_slice(name=slice_name)

n1 = slice.add_node(name="n1", image=image, cores=2, ram=4, disk=9, site=site)
n2 = slice.add_node(name="n2", image=image, cores=2, ram=4, disk=9, site=site)
n3 = slice.add_node(name="n3", image=image, cores=2, ram=4, disk=9, site=site)
n4 = slice.add_node(name="n4", image=image, cores=2, ram=4, disk=9, site=site)
n5 = slice.add_node(name="n5", image=image, cores=2, ram=4, disk=9, site=site)
n6 = slice.add_node(name="n6", image=image, cores=2, ram=4, disk=9, site=site)
n7 = slice.add_node(name="n7", image=image, cores=2, ram=4, disk=9, site=site)
n8 = slice.add_node(name="n8", image=image, cores=2, ram=4, disk=9, site=site)
n9 = slice.add_node(name="n9", image=image, cores=2, ram=4, disk=9, site=site)

iface1 = n1.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface2 = n2.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface3 = n3.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface4 = n4.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface5 = n5.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
```

```

iface6 = n6.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface7 = n7.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface8 = n8.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface9 = n9.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface10 = n10.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface11 = n11.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]
iface12 = n12.add_component(model="NIC_Basic", name="nic1").get_interfaces()[0]

net = slice.add_l2network(name="trading_net", interfaces=[iface1, iface2, iface3, iface4,
                                                         iface5, iface6, iface7, iface8, iface9,
                                                         iface10, iface11, iface12
                                                         ])

slice.submit()

```

```

ips = {
    "n1": "192.168.1.1",
    "n2": "192.168.1.2",
    "n3": "192.168.1.3",
    "n4": "192.168.1.4",
    "n5": "192.168.1.5",
    "n6": "192.168.1.6",
    "n7": "192.168.1.7",
    "n8": "192.168.1.8",
    "n9": "192.168.1.12"
}

for node in slice.get_nodes():
    node_name = node.get_name()
    node_ip = ips[node_name]

    iface = node.get_interface(network_name="trading_net")
    os_iface_name = iface.get_os_interface()

    print(f"Configuring {node_name} (Interface: {os_iface_name})...")

    node.execute(f"sudo ip addr flush dev {os_iface_name}")
    node.execute(f"sudo ip addr add {node_ip}/24 dev {os_iface_name}")
    node.execute(f"sudo ip link set dev {os_iface_name} up")

```

```
print(f'{node_name} IPv4: {node_ip}\n')
```

```
from ipaddress import ip_address, IPv6Address

try:
    n1 = slice.get_node(name="n1")

    if type(ip_address(n1.get_management_ip())) is IPv6Address:

        commands = (
            "sudo unlink /etc/resolv.conf || true && "
            "echo 'nameserver 2a00:1098:2c::1' | sudo tee /etc/resolv.conf && "
            "echo 'nameserver 2a01:4f8:c2c:123f::1' | sudo tee -a /etc/resolv.conf && "
            "echo 'nameserver 2a00:1098:2b::1' | sudo tee -a /etc/resolv.conf && "
            "echo '\n complete'"
        )

        stdout, stderr = n1.execute(commands)

        print(stdout)

        if stderr:
            print(stderr)

except Exception as e:
    print(f"Exception: {e}")
```

## 2. Installing Dependencies

Once the nodes are active, SSH into each node to install system requirements, Python, and project dependencies:

```
sudo apt update
sudo apt install -y python3-pip python3-dev gcc make python-is-python3
git clone https://github.com/afoong2003/cs451_project.git
cd cs451_project
pip install --no-cache-dir -r requirements.txt
```

#### 4. Running the System

1. On n9:

```
python3 execution.py
```

2. On n2-n8:

```
python3 decision_node.py FAKEPACA
```

3. On n1:

```
python3 market_data.py
```

### Limitations of my implementation

Currently, my implementation relies on a specific environment where the market has a clear upward or downward trend. If a sideways market were to occur, my implementation will not be able to make accurate decisions. A sideways market is when the price stays in a small range and stays in that range for a long period. Sideways market can make my strategy consistently sell an asset for a small loss each time. Another limitation is having RSI less than 70 to trigger a buy. For example if an asset were to be overbought but the price is still trending upwards, my implementation will not be able to capitalize on this scenario.

### What I'll add if I had 3 more months to work on this project

If I had 3 more months, I would implement a system to detect a sideways market. When a sideways market occurs, it will switch to a different strategy or completely stop using the strategy with RSI and SMA. Also adding another condition to check trading volume before buying. I could also implement a live dashboard with graphs detailing sma, rsi, and other features. And do more testing, document which strategy is successful. If I were to do calculations for 2 assets per node, it could improve resource efficiency and reduce network calls.